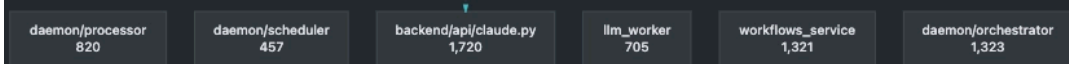


AI Agent Architecture

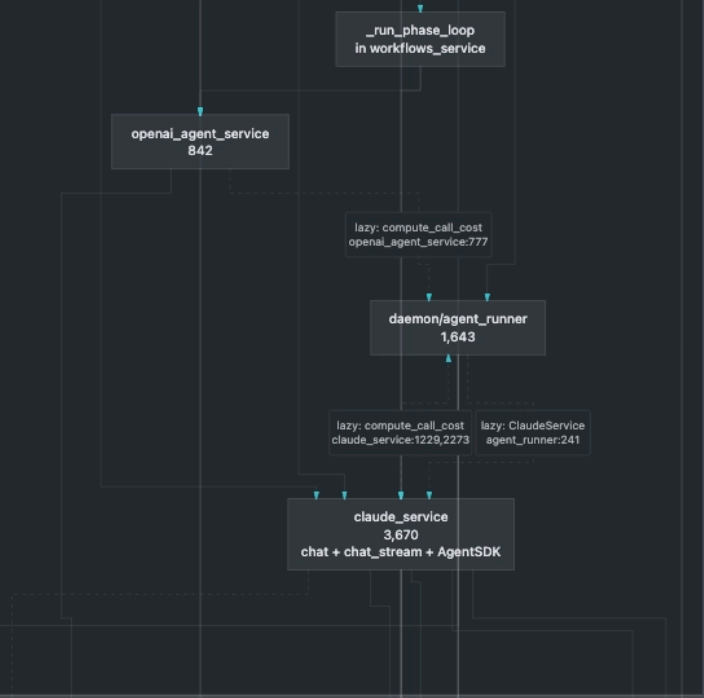
Proposal: rearchitecting ai agent architecture to make it modular and be able to implement a vast amount of tools, agents, mcp servers, context layer as we expand vigil out.

State of architecture (7/27):

entry points

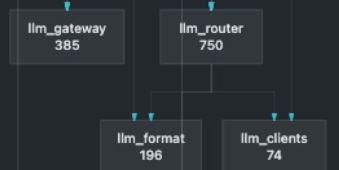


FIVE agentic loops

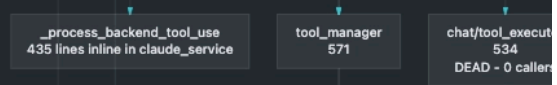


lazy: backend.api.mcp
claude_service:3526

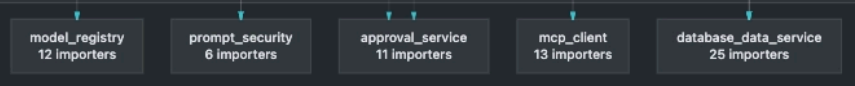
transport



THREE tool dispatchers



shared hubs

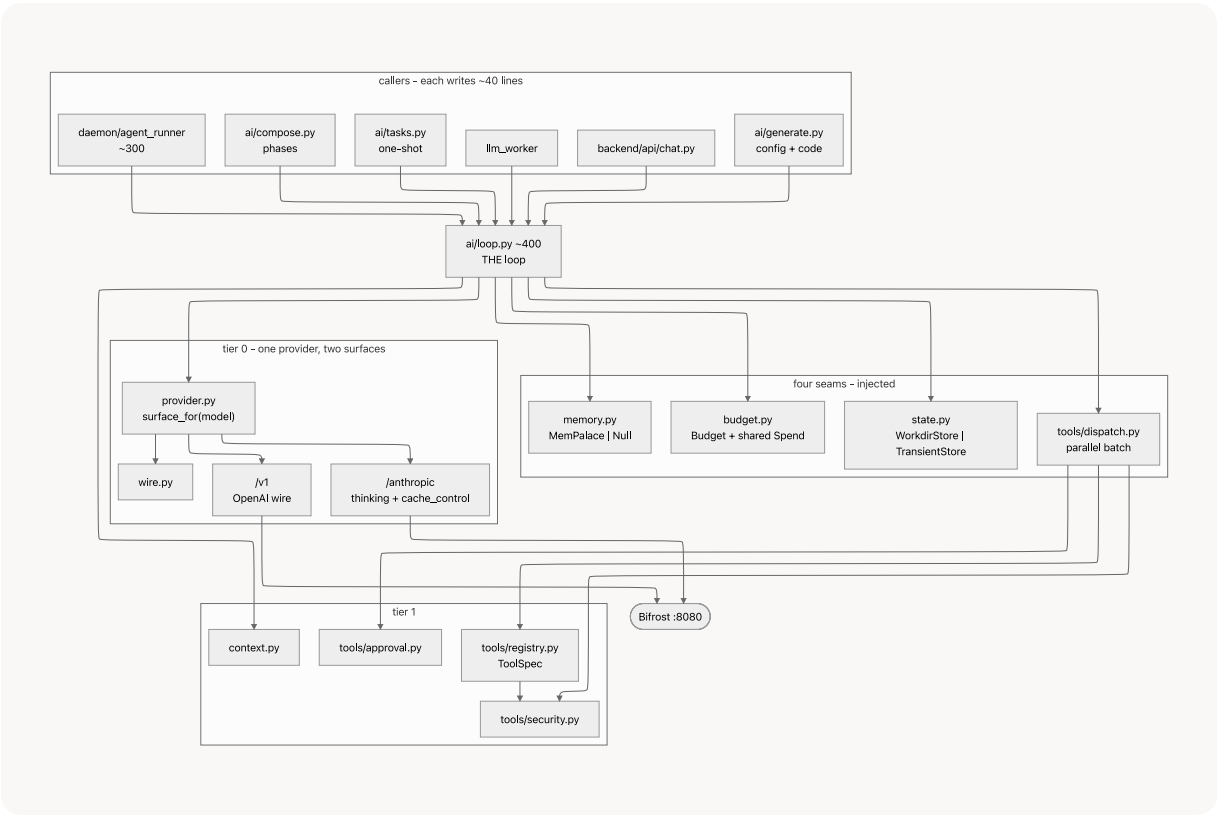


Key features/issues as of right now:

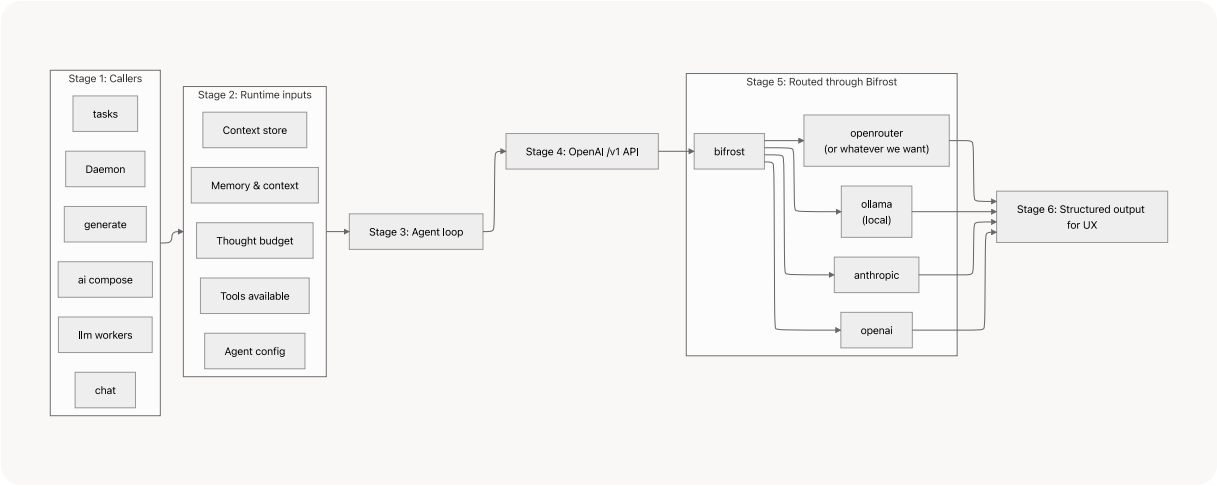
- Rate Limiting duplicated across three different modules (deeptempo-core), bifrost, claude_service.py
- 5 different agent loops for different objectives
 - open ai agent service
 - daemon
 - claude service
 - run phase loop
 - daemon/agent_runner
- 3 tool dispatchers
 - process_backend_tool_use, tool manager, chat/tool_executor
- Several transport layers
 - llm_gateway, llm_router, llm_format, llm_clients

Proposed architecture:

Nuanced look



More simply:



Agent Loop

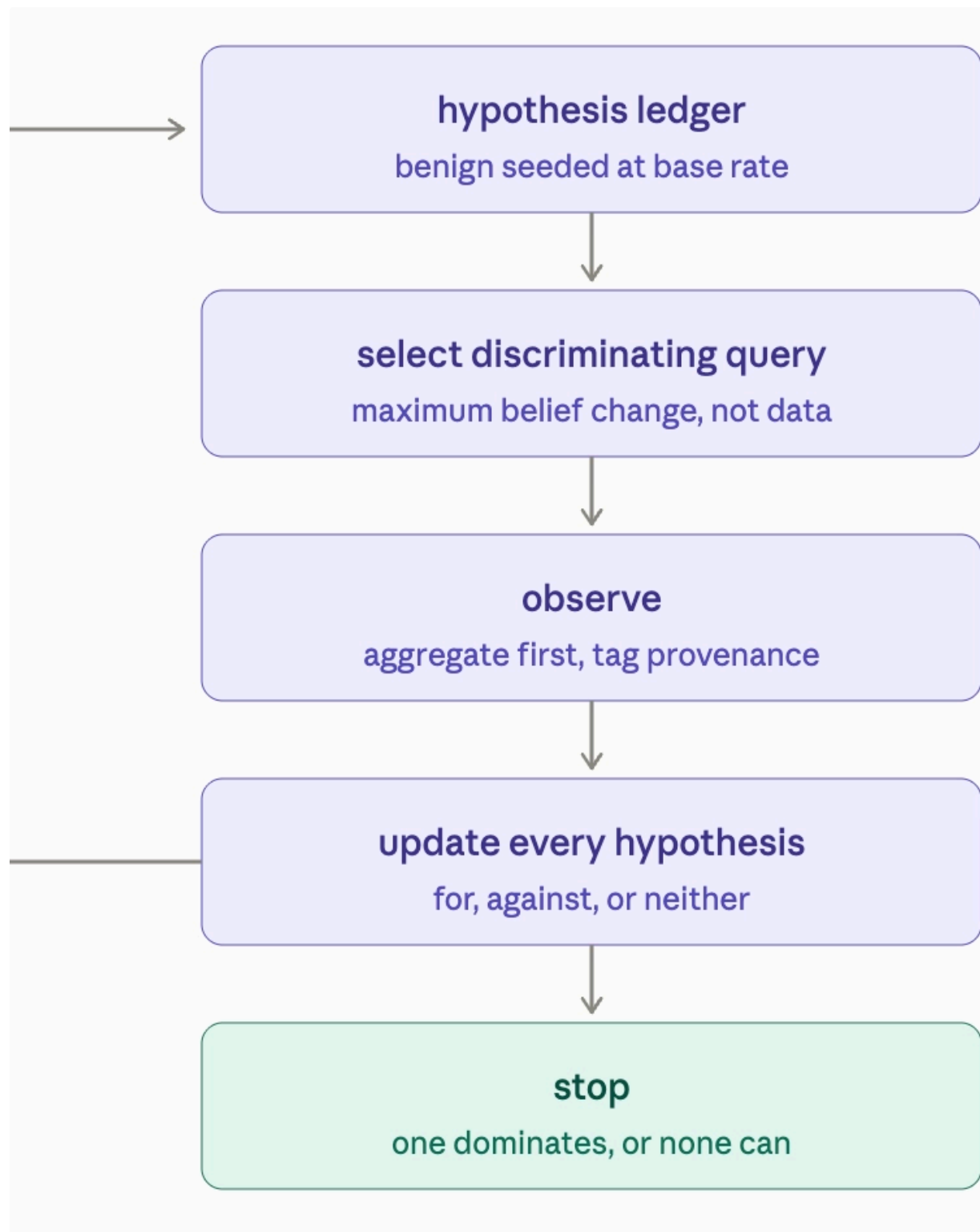
This is what the loop itself should look like:

We want a nuanced ReAct specific loop for a few key reasons:

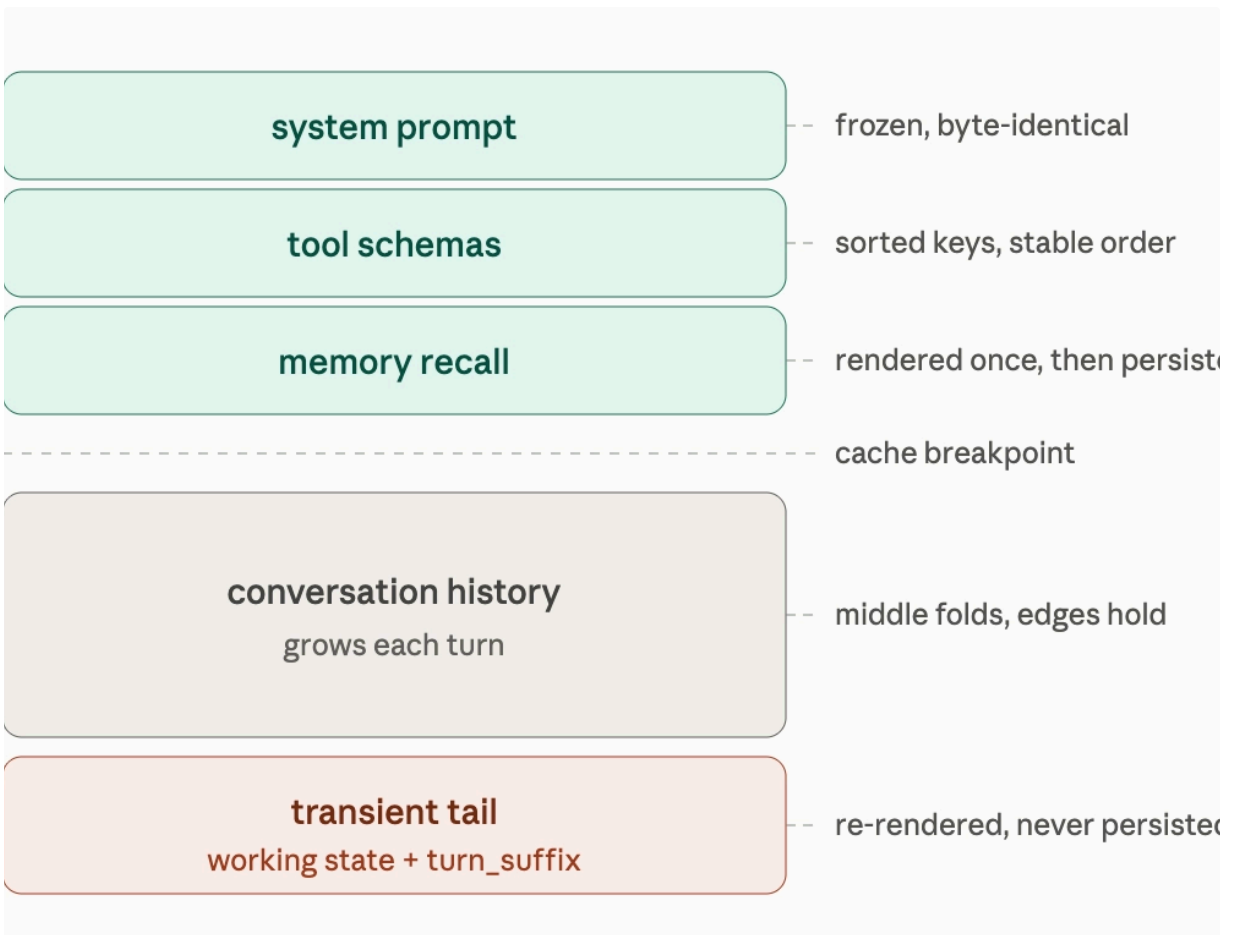
- *What if lateral movement or a certain investigation doesn't exist?*
 - An LLM with a hardcoded objective will always search for a result, even if it doesn't exist by nature of employing an llm + tools to loop until it accomplishes a task (ie coding agent)
 - We would rather it fail and NOT know the answer, than give an inaccurate answer:
 - From JVL:

"The reason I am having this conversation is because Vigil has a lot of genuinely good infrastructure (the case management layer, the approval workflow, the cost tracking) that I do not want to rebuild. But my boss Marty set a bar: if the AI output is not trustworthy, it is worse than not having it at all. So the prompt/agent control of the entire workflow question is the critical one for us."
- *What if a threat isn't malicious?*
 - Confirmation drift
- *Greedy querying*
 - An LLM will query something based upon a set belief set by system prompt, and if it changes throughout the investigation, what happens?

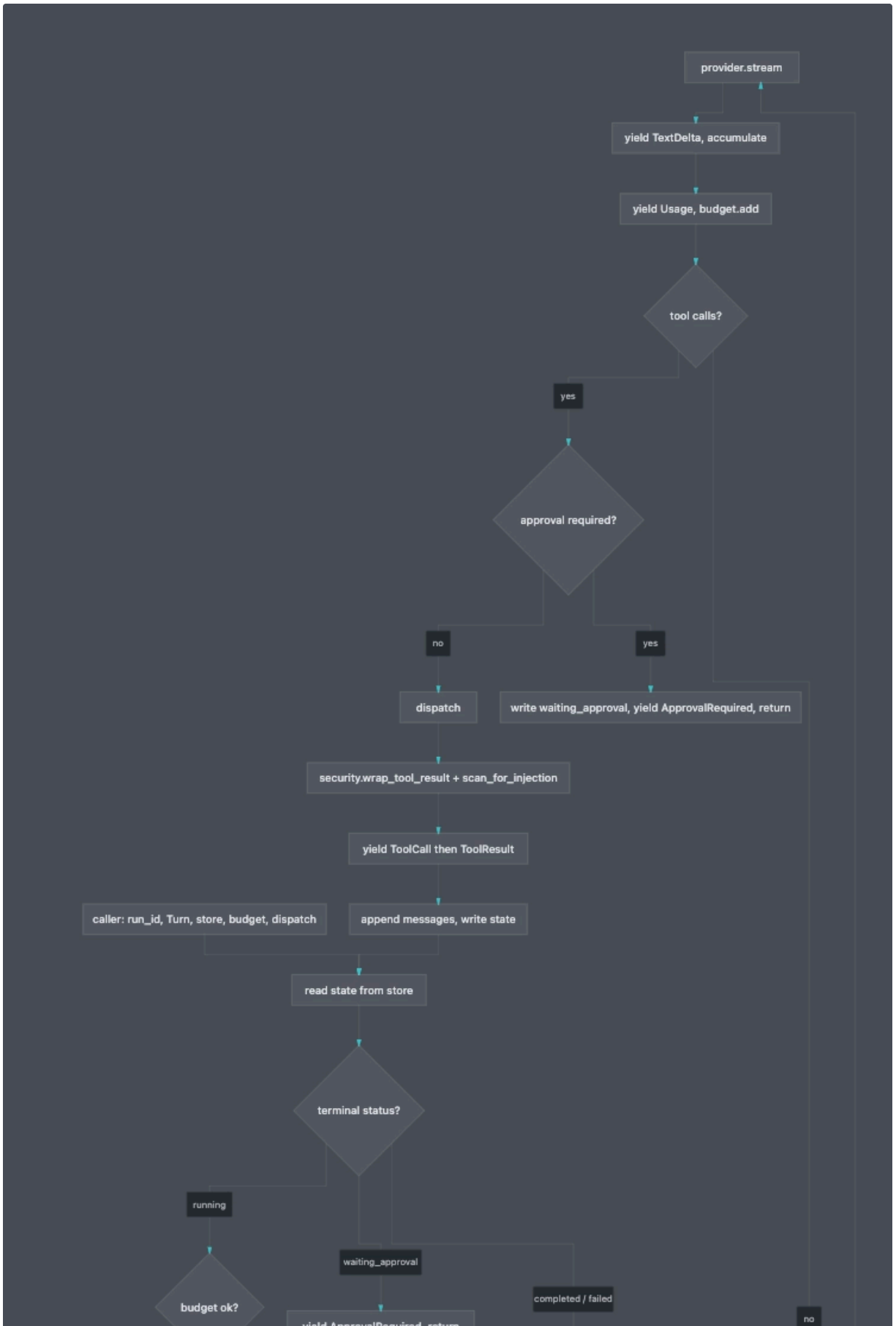
What I propose, which in nature is a ReAct harness with key dynamic qualification criteria:

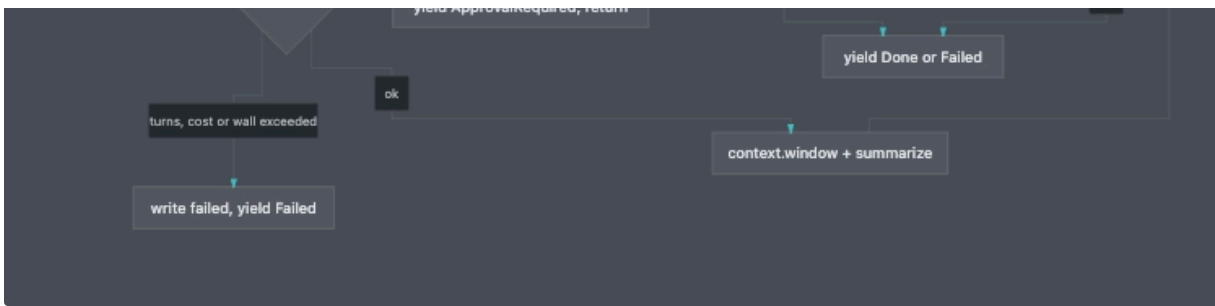


Standard context breakdown:



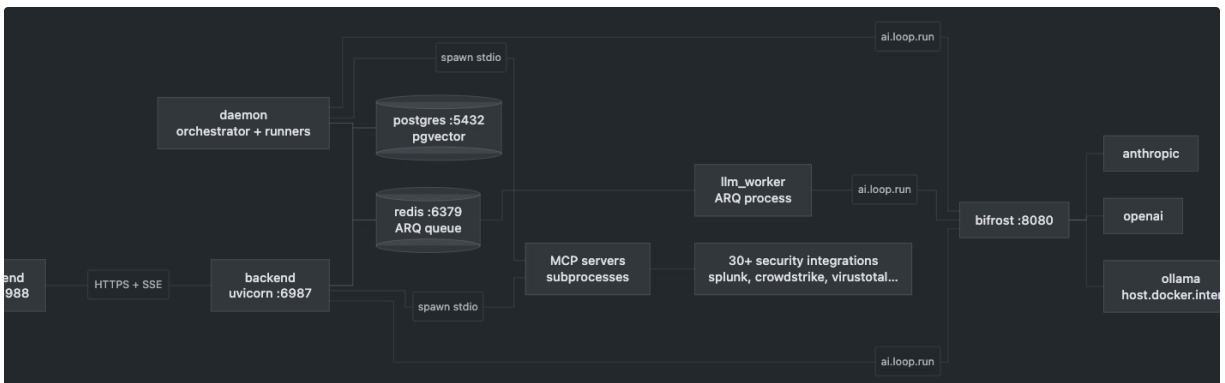
Nuanced deterministic workflow attached to general harness





- By having a belief that can be changed, such as a hypothesis as opposed to "this is malicious, investigate" we allow for flexibility. In investigations specifically, this loop is highly effective and can be placed as the overarching

Quick look at what this looks like in the scope of the entire vigil ecosystem:



Key questions: @Mayank Kumar

- how do we handle subagents
Mayank: we might need to design orchestration for open-source models.
- how do we handle memory? memplace application is brittle
- how can we be mindful of a context layer? we keep talking about this , and while this architecture would integrate well with that, I don't know what it looks like and how it would integrate with the architecture

- how can we dynamically adjust tool availability + costs? (ie defining a wide variety of tools uses a lot of tokens and while good for functionality/versatility, can use a resources unnecessarily)
- what about flexible schemas (flowprep)?
 - histogram/time series analysis agent
- building out in the open vs community driven - Mayank: building out in open.
- state on RDDs for future direction + alignment